

B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha

`brejcmic@fel.cvut.cz`

ČVUT v Praze, FEL

Praha, 2025

Obsah

- 1 Sériová komunikace
- 2 ASCII driver - Control Web

Téma

- 1 Sériová komunikace
- 2 ASCII driver - Control Web

Sériová komunikace

- způsob přenosu dat, při kterém se informace posílají postupně po jednotlivých bitech za sebou,
- jako komunikační kanál v jednom směru často dostačuje jen jeden vodič.

Základní princip:

- Odeslání binární hodnoty $(10110010)_2$,
- na komunikační kanál vysílač postupně se stanovenou periodou zadává hodnoty:

$1 \longrightarrow 0 \longrightarrow 1 \longrightarrow 1 \longrightarrow 0 \longrightarrow 0 \longrightarrow 1 \longrightarrow 0$

- přijímač i vysílač musí nějakým způsobem stanovit časový interval pro vyslání a čtení hodnoty v komunikačním kanálu.

Sériová komunikace - výhody

- méně vodičů,
- jednodušší zapojení,
- levnější hardware,
- možnost komunikace na větší vzdálenosti,
- menší citlivost na rušení.

Poslední dva body platí především pro linky využívající proudovou smyčku nebo diferenciální pár.

Rozlišujeme:

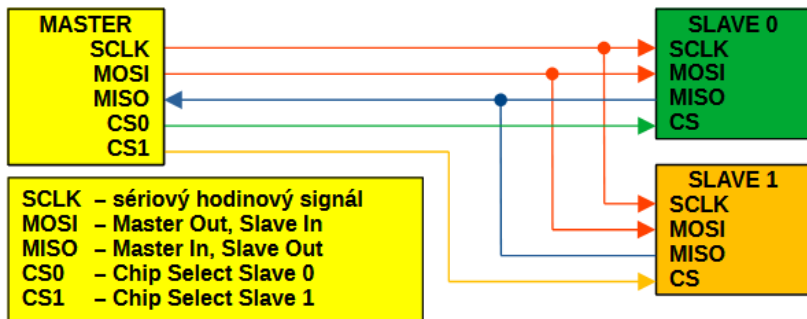
- **synchronní** komunikaci - zařízení sdílí společný hodinový signál (SPI, I²C),
- **asynchronní** komunikaci - bez sdíleného hodinového signálu (RS232, RS485).

SPI

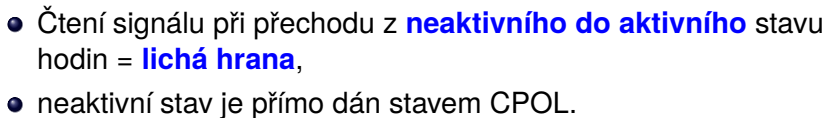
Serial Peripheral Interface

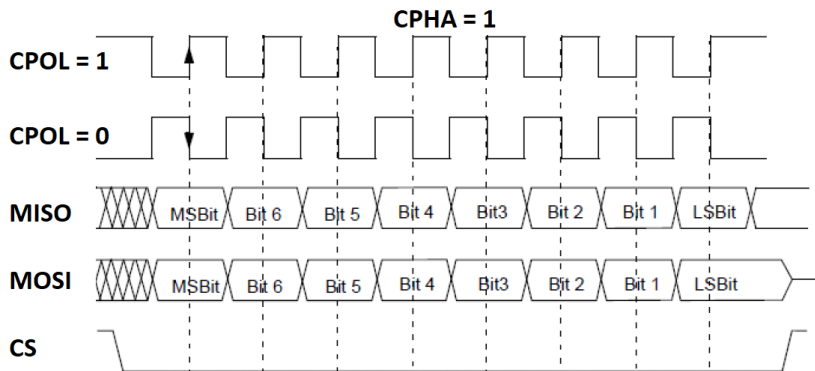
- komunikace s paměťmi a senzory,
- jeden obvod je řídicí - Master a vystavuje řídicí signály komunikace,
- ostatní obvody, jsou typu Slave,
- Full-Duplex komunikace - oběma směry se komunikuje v jeden moment,
- je určena pro kratší komunikační vzdálenosti,
- čtení i vyslání bitu je řízeno hodinovým signálem, který je součástí komunikace,
- rychlost komunikace definuje Master pomocí hodinového signálu a je omezena jen fyzikálními limity komunikační linky.

SPI - zapojení



- Lze komunikovat s více Slave jednotkami na jedné sběrnici, pokud existuje příslušný CS signál.

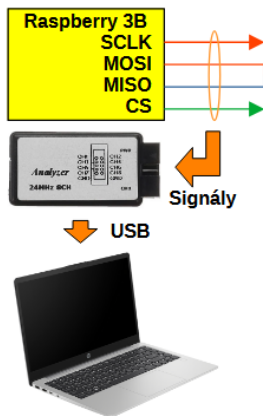


$$\text{SPI} - \text{CPHA} = 1$$


- Čtení signálu při přechodu z **aktivního do neaktivního** stavu hodin = **sudá hrana**,
- neaktivní stav je přímo dán stavem CPOL.

SPI - ukázka, loopback

Zapojení:

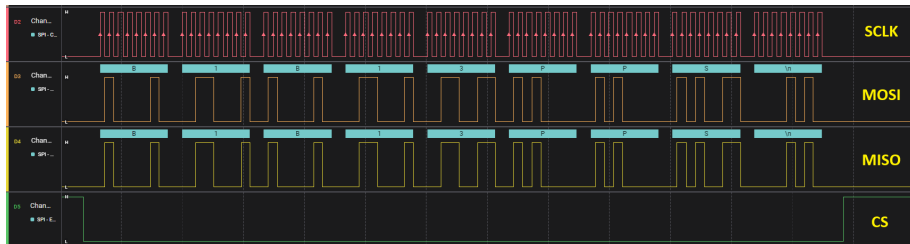


RPI program:

```

11 import spidev
12
13 class SPIExample:
14     """
15     Objekt ukázkového příkladu SPI komunikace
16     """
17     def __init__(self):
18         """
19         Nastavení SPI komunikace
20         """
21         self._dev = spidev.SpiDev()
22         self._dev.open(bus=0, device=1)
23         self._dev.max_speed_hz = 100000
24         self._dev.mode = 0b00
25
26     def __del__(self):
27         """
28         Uzavření komunikace
29         """
30         if self._dev is not None:
31             self._dev.close()
32
33     def transaction_call(self):
34         int_list = list(b'B1B13PPS\n')
35         response = self._dev.xfer2(int_list)
36         print("Přijata zpráva: {}".format(bytes(response).decode('ascii')))
37
38
39 if __name__ == '__main__':
40     obj = SPIExample()
41     obj.transaction_call()
42 
```

SPI - logický analyzátor



- Znaky jsou kódovány pomocí ASCII,
- CPOL=0, klidová hodnota hodin je 0
- CPHA=0, lichá hrana, přechod z 0 do 1

ASCII tabulka

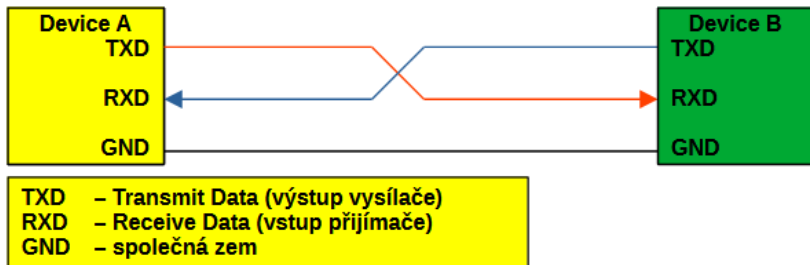
Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value
00	NUL	10	DLE	20	SP	30	0	40	@	50	P	60	`	70	p
01	SOH	11	DC1	21	!	31	1	41	A	51	Q	61	a	71	q
02	STX	12	DC2	22	"	32	2	42	B	52	R	62	b	72	r
03	ETX	13	DC3	23	#	33	3	43	C	53	S	63	c	73	s
04	EOT	14	DC4	24	\$	34	4	44	D	54	T	64	d	74	t
05	ENQ	15	NAK	25	%	35	5	45	E	55	U	65	e	75	u
06	ACK	16	SYN	26	&	36	6	46	F	56	V	66	f	76	v
07	BEL	17	ETB	27	'	37	7	47	G	57	W	67	g	77	w
08	BS	18	CAN	28	(	38	8	48	H	58	X	68	h	78	x
09	HT	19	EM	29	)	39	9	49	I	59	Y	69	i	79	y
0A	LF	1A	SUB	2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
0B	VT	1B	ESC	2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
0C	FF	1C	FS	2C	,	3C	<	4C	L	5C	\	6C	l	7C	
0D	CR	1D	GS	2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
0E	SO	1E	RS	2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
0F	SI	1F	US	2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

UART

Universal Asynchronous Receiver-Transmitter

- Základ pro:
 - standard RS232, RS422: komunikace mezi dvěma rovnocennými zařízeními,
 - standard RS485: komunikace mezi mnoha zařízeními (**Half-Duplex**)
- počátky v 60. letech (1969 RS232C),
- formát se stále používá v průmyslu - Modbus RTU, M-Bus, ...
- základem je obousměrná komunikace po 2/3 vodičích,
- RS232 používá navíc řídicí vodiče RTS, CTS, DTR, RI, DCD,
- standardy se liší v napěťových úrovních:
 - **RS232**: log. 0: -15V až -3V, log. 1: 3V až 15V
 - **TTL**: log. 0: 0V, log. 1: 5V

UART - zapojení

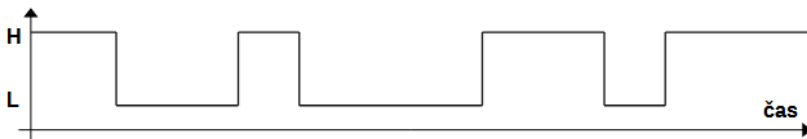
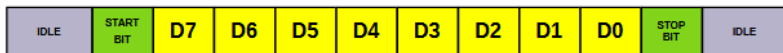


- Bez vodiče sdíleného hodinového signálu:
 - musí dojít k synchronizaci přijímače s vysílanými daty,
 - se musí shodovat přenosová rychlost,
 - se musí shodovat rámec přenášených dat (např. počet bitů).

UART - datový rámeček

Rámeček pro 8 datových bitů, 1 stop bit a bez parity:

Přenášený znak: „F“ (ASCII 0x46 = 01000110₂)



START BIT	Označuje začátek rámečku - hrana
D7:0	Data, pořadí MSB → LSB
STOP BIT	Linka je v klidové úrovni po dobu 1 bitu

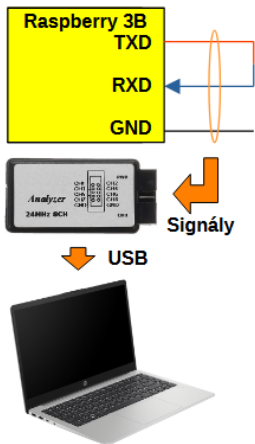
UART - vlastnosti

- **Baud (Bd)**: jednotka modulační rychlosti, počet změn stavu přenosového média za sekundu,
- standardizované rychlosti: 150, 300, 600, 1200, 2400, 4800, 9600, 19200, 38400, 57600 a 115200 Bd
- vždy 1 start bit,
- pořadí dat:
 - **MSB→LSB**
 - **LSB→MSB**
- možnost paritního bitu:
 - sudá parita doplňuje počet 1 na sudý počet,
 - lichá parita doplňuje počet jedniček na lichý počet,
- 1, 1.5 nebo 2 stop bity.

UART - ukázka, loopback

Zapojení:

RPI program:

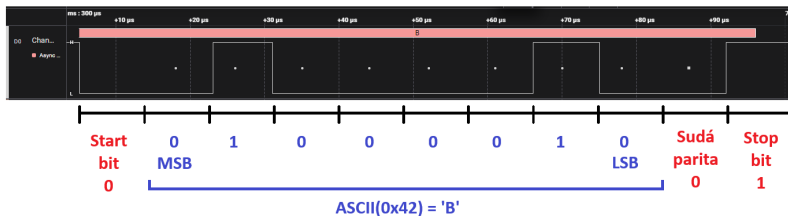


```

11 import serial
12
13 class SerialExample:
14     """
15     Objekt ukázkového příkladu sériové komunikace
16     """
17     def __init__(self):
18         """
19         Nastavení sériové komunikace
20         """
21         self.__dev = serial.Serial(port='/dev/serial0',
22                                   baudrate=115200,
23                                   parity=serial.PARITY_EVEN,
24                                   stopbits=serial.STOPBITS_ONE,
25                                   timeout=2)
26         self.__bytes_cnt = 0
27
28     def __del__(self):
29         """
30         Uzávěření komunikace
31         """
32         if self.__dev is not None and self.__dev.is_open:
33             self.__dev.close()
34
35     def transaction_call(self):
36         self.__bytes_cnt = self.__dev.write(b'B1B13PPS\n')
37         self.__dev.flush()
38         response = self.__dev.read(size=self.__bytes_cnt)
39         print("Přijata zpráva: {}".format(response))
40
41
42 if __name__ == '__main__':
43     obj = SerialExample()
44     obj.transaction_call()

```

UART - logický analyzátor



- Jednodušší komunikace než SPI,
- formát přenosu musí být nastaven dopředu na obou stranách.

Téma

1 Sériová komunikace

2 ASCII driver - Control Web

UART v PC

- Moderní PC nemají port RS232 ani UART,
- v praxi se používají převodníky pro USB.

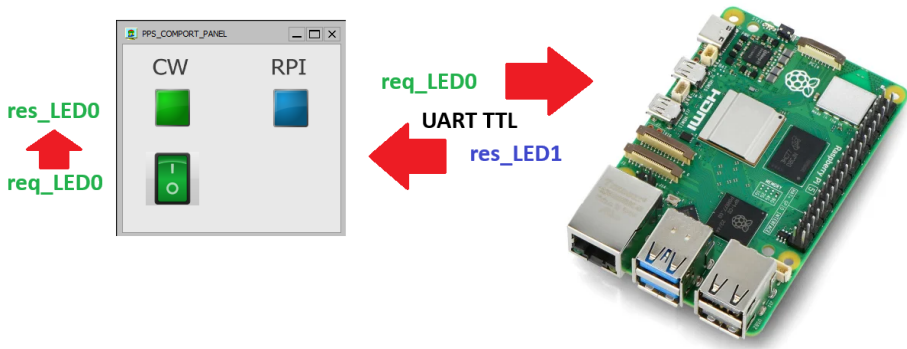
USB-UART-TTL



USB-RS232

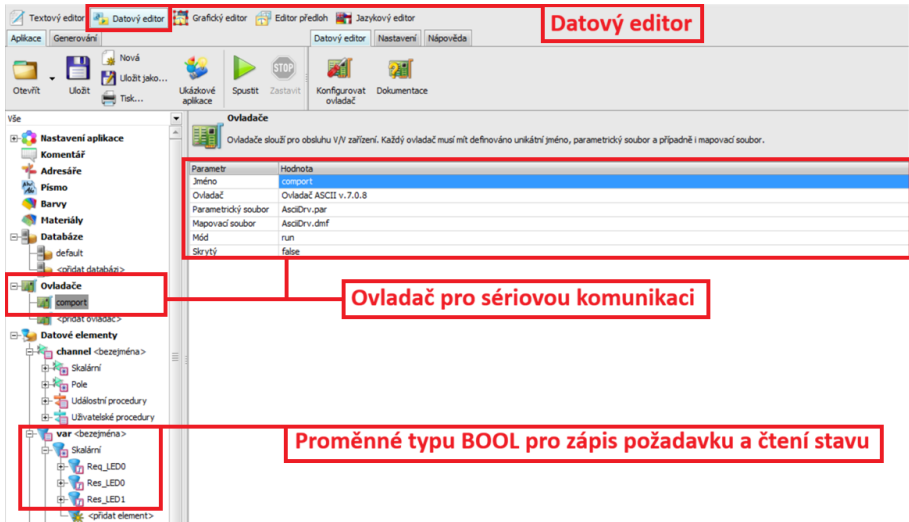


Komunikace - požadavky



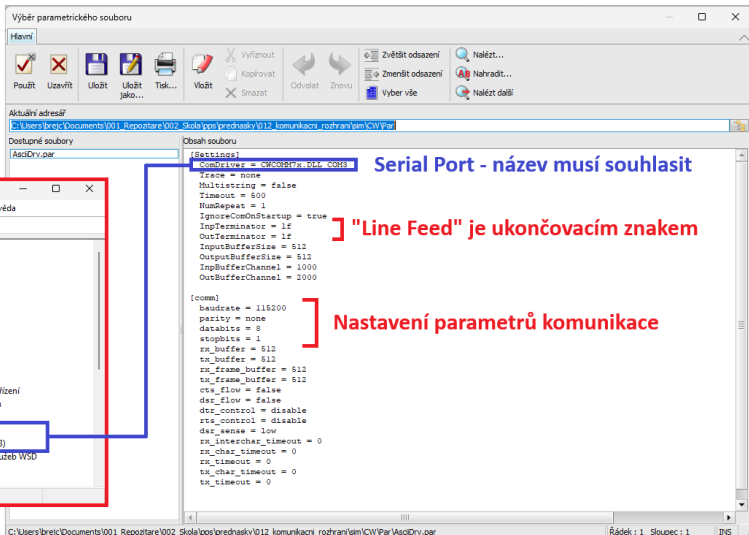
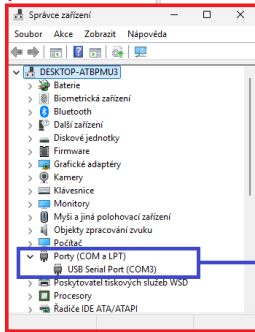
- Požadavek **req_LED0** změní stav zelené led a zároveň je odeslán do RPI, které odpoví stejným stavem modré LED.
- **Nastavení sériové komunikace:** 115200 Bd, 8 bit, 1 stop bit, bez parity.
- **ASCII zprávy:** „**LEDxH**“ pro log 1, „**LEDxL**“ pro log 0.

Příprava proměnných a ovladače



Parametrický soubor ovladače

Správce zařízení



Kanály ovladače

 Kanály Kanál slouží jako vazba mezi aplikací a V/V zařízením. Vlastní komunikaci s V/V zařízením zprostředkovává ovladač.								
name	type	init_value	driver	driver_index	direction	timeout	comment	
 StringRx	string		comport	20	input		'přijátý řetězec'	
 StringRxLen	real		comport	21	input		'délka přijatého řetězce'	
 StringTx	string		comport	22	output		'vyslaný řetězec'	
 StringTxLen	real		comport	24	output		'délka vyslaného řetězce'	
 <přidat>	<přidat>	<přidat>	<přidat>	<přidat>	<přidat>	<přidat>	<přidat>	

Pro posílání a přijímání zpráv si vystačíme jen se dvěma kanály:

- **Kanál 22:** sem zapsaný řetězec se okamžitě vyšle po sériové lince, nečeká se na odpověď,
- **Kanál 23:** sem zapsaný řetězec se okamžitě vyšle po sériové lince, ale čeká se na odpověď,
- **Kanál 20:** reprezentuje poslední přijatý řetězec.

Alternativně je možné používat přímá volání procedur ovladače předpisem:

- `core.DriverQueryProc('driver', 'procedura', hodnota)`

Program

ControlWeb

```

procedure OnActivate();
begin

    // vyhodnoceni stavu zelene LED
    if Res_LED0 <> Req_LED0 then
        if Req_LED0 then
            StringTx = 'LED0H';
        else
            StringTx = 'LED0L';
        end;
    end;

    Res_LED0 = Req_LED0;

    // vyhodnoceni stavu modre LED
    if StringRx = 'LED1H' then
        Res_LED1 = true;
    else
        Res_LED1 = false;
    end;

    // povoleni vyjimek pro dalsi prijem
    core.DriverQueryProc( 'comport', 'EnableException', 0 );
end_procedure;

```

RPI

```

11 import serial
12
13 class AsciiDriver:
14     """
15     Objekt ukázkového příkladu sériové komunikace
16     """
17     def __init__(self):
18         """
19         Nastavení sériové komunikace
20         """
21         self.__dev = serial.Serial(port="/dev/serial0",
22                                   baudrate=115200,
23                                   parity=serial.PARITY_NONE,
24                                   stopbits=serial.STOPBITS_ONE,
25                                   timeout=2)
26
27         self.__bytes_cnt = 0
28
29     def __del__(self):
30         """
31         Uzavření komunikace
32         """
33         if self.__dev is not None and self.__dev.is_open:
34             self.__dev.close()
35
36     def transaction_call(self):
37         response = self.__dev.readline() # čekání na řetězec
38         if len(response) > 0:           # kontrola příjmu
39             print(response)             # tisk řetězce
40             if response == b'LED0H\n':  # porovnání řetězce se zprávou
41                 print("Green ---> Blue [XX]")
42                 self.__dev.write(b'LED1H\n')
43                 self.__dev.flush()
44             else:
45                 print("Green ---> Blue [ ]")
46                 self.__dev.write(b'LED1L\n')
47                 self.__dev.flush()
48
49     if __name__ == '__main__':
50         obj = AsciiDriver()
51
52         while True:
53             obj.transaction_call()

```

Logický analyzátor



- Mezi příjmem požadavku a odpovědí je 1 ms zpoždění,
- každá zpráva trvá přibližně 500 μ s,
- celkové minimální zpoždění reakce systému je kolem 2 ms,
- UART je jednoduché a snadno implementovatelné rozhraní, ale kvůli nižší přenosové rychlosti a režii rámců není vhodný pro velmi rychlé nebo časově kritické aplikace.

Srovnání UART a SPI

Vlastnost	UART	SPI
Typ komunikace	asynchronní	synchronní
Počet vodičů	2 + GND	minimálně 4
Směr komunikace	full-duplex	full-duplex
Topologie	bod–bod	master–slave
Přenosová rychlost	nižší	velmi vysoká
Režie přenosu	start/stop bity	minimální
Adresace zařízení	ne	přes CS signály
Více zařízení	komplikované	snadné
Délka vedení	větší	spíše kratší
Implementace	velmi jednoduchá	složitější
Typické použití	terminály, SCADA, GPS	senzory, ADC, Flash

Následující přednáška

- Protokol MODBUS RTU

Děkuji za pozornost